

Examen Practico

Nombre: Jeremy Ruperto Gaibor Rodríguez

INDICE

INTRODUCCIÓN	2
DESARROLLO	2
1. Creación del proyecto	2
2. Configuración del archivo pom.xml	2
3. Evidencia de la estructura del proyecto	4
4. Creación de las clases comunes	4
4.1. Nodo.java	4
4.2. Mensaje.java	5
4.3. RelojLamport.java	6
4.4. CsvLogger.java	7
4.5. AuthToken.java	8
5. Creación de la comunicación TCP	8
5.1. ServidorTCP.java	8
5.2. ClienteTCP.java	16
6. Creación de los nodos distribuidos	17
6.1. Nodo1TCP.java	17
6.2. Nodo2TCP.java	17
6.3. Nodo3TCP.java	18
7. Evidencia de ejecución	18
7.1. Ejecución de los tres nodos	18
7.2. Prueba de caída del coordinador	18
7.3. Elección del nuevo coordinador N2	19
7.4. Evidencia del archivo ANALISIS.md	19
CONCLUSIÓN	19

INTRODUCCIÓN

El presente documento evidencia la creación de un prototipo de sistema distribuido desarrollado en Java con Maven, donde la implementación se basa en tres nodos TCP que se comunican entre sí mediante sockets, utilizando mensajes en formato JSON.

El prototipo incluye clases comunes para representar nodos, mensajes y reloj lógico de Lamport. Además, incorpora heartbeats para detectar fallos, el algoritmo Bully para elegir un nuevo coordinador y un registro CSV para guardar los eventos generados durante la ejecución.

Las evidencias presentadas muestran la estructura del proyecto, las clases creadas y las pruebas realizadas para comprobar la comunicación, la detección de caída del coordinador y la elección de un nuevo líder.

DESARROLLO

1. Creación del proyecto

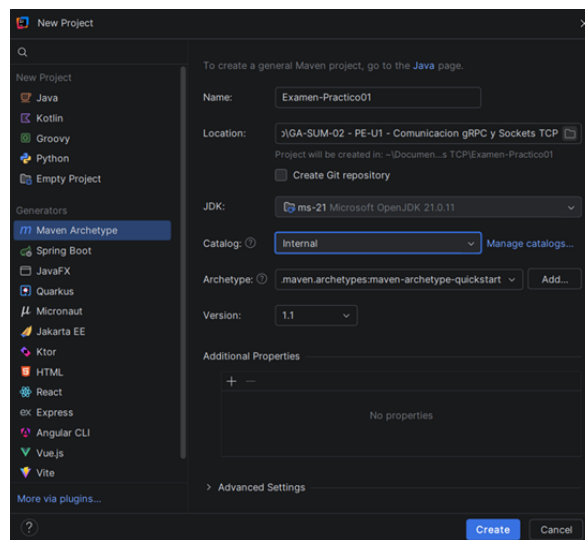


Figura 1: Nuevo proyecto

2. Configuración del archivo pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
```

```

<groupId>org.uteq</groupId>
<artifactId>Examen-Practico01</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>

<name>Examen-Practico01</name>
<url>http://maven.apache.org</url>

<properties>
  <project.build.sourceEncoding>UTF-8</project.build.
    sourceEncoding>
</properties>

<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>

  <!-- JSON para Sockets TCP -->
  <dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.11.0</version>
  </dependency>

</dependencies>
</project>

```

3. Evidencia de la estructura del proyecto

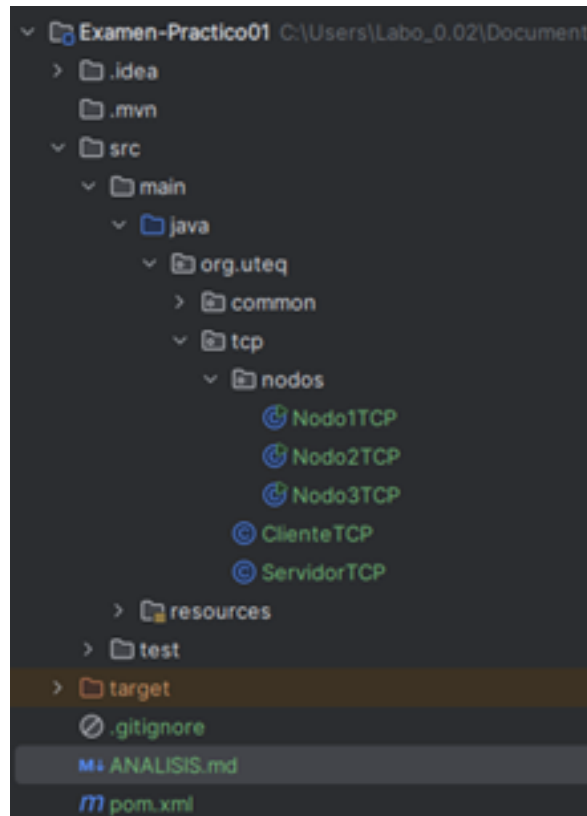


Figura 2: Estructura del proyecto Maven

4. Creación de las clases comunes

4.1. *Nodo.java*

Esta clase define la información de cada nodo del sistema distribuido. Contiene el identificador, nombre y puerto TCP de Nodo1, Nodo2 y Nodo3. También permite manejar la lista de todos los nodos disponibles.

```
1 package org.uteq.common;
2
3 public class Nodo {
4
5     private final int id;
6     private final String nombre;
7     private final int puertoTcp;
8
9     public static final Nodo NOD01 = new Nodo(1, "Nodo1", 5000);
10    public static final Nodo NOD02 = new Nodo(2, "Nodo2", 5001);
11    public static final Nodo NOD03 = new Nodo(3, "Nodo3", 5002);
12
```

```

13     public static final Nodo[] TODOS = {NOD01, NOD02, NOD03};
14
15     public Nodo(int id, String nombre, int puertoTcp) {
16         this.id = id;
17         this.nombre = nombre;
18         this.puertoTcp = puertoTcp;
19     }
20
21     public int getId() {
22         return id;
23     }
24
25     public String getNombre() {
26         return nombre;
27     }
28
29     public int getPuertoTcp() {
30         return puertoTcp;
31     }
32 }

```

4.2. Mensaje.java

Esta clase representa la estructura de los mensajes enviados entre nodos. Incluye el tipo de mensaje, nodo emisor, identificador, timestamp Lamport, contenido y token de validación.

```

1  package org.uteq.common;
2
3  public class Mensaje {
4
5      private String tipo;           // OPERACION, HEARTBEAT, ELECCION,
                                     COORDINADOR, ACK, ERROR
6      private String sender;
7      private int nodoId;
8      private int timestamp;
9      private String message;
10     private String token;
11
12     public Mensaje() {
13     }
14
15     public Mensaje(String tipo, String sender, int nodoId, int
        timestamp, String message, String token) {
16         this.tipo = tipo;
17         this.sender = sender;
18         this.nodoId = nodoId;

```

```

19         this.timestamp = timestamp;
20         this.message = message;
21         this.token = token;
22     }
23
24     public String getTipo() {
25         return tipo;
26     }
27
28     public String getSender() {
29         return sender;
30     }
31
32     public int getNodoId() {
33         return nodoId;
34     }
35
36     public int getTimestamp() {
37         return timestamp;
38     }
39
40     public String getMessage() {
41         return message;
42     }
43
44     public String getToken() {
45         return token;
46     }
47 }

```

4.3. RelojLamport.java

Esta clase implementa el reloj lógico de Lamport. Permite incrementar el tiempo local cuando se envía un mensaje y actualizarlo cuando se recibe un timestamp de otro nodo.

```

1 package org.uteq.common;
2
3 public class RelojLamport {
4
5     private int tiempo;
6
7     public RelojLamport() {
8         this.tiempo = 0;
9     }
10
11     public synchronized int incrementar() {

```

```

12         tiempo++;
13         return tiempo;
14     }
15
16     public synchronized int actualizar(int recibido) {
17         tiempo = Math.max(tiempo, recibido) + 1;
18         return tiempo;
19     }
20
21     public synchronized int getTiempo() {
22         return tiempo;
23     }
24 }

```

4.4. CsvLogger.java

Esta clase registra los eventos del sistema en un archivo CSV. Guarda información como timestamp, nodo, tipo de evento y detalle, permitiendo evidenciar mensajes, fallos y cambios de coordinador.

```

1  package org.uteq.common;
2
3  import java.io.File;
4  import java.io.FileWriter;
5  import java.io.IOException;
6  import java.io.PrintWriter;
7
8  public class CsvLogger {
9
10     public static void guardarEvento(String archivo, int timestamp
11         , int nodoId, String nodo, String tipo, String detalle) {
12         File file = new File(archivo);
13         boolean archivoNuevo = !file.exists();
14
15         try {
16             File carpeta = file.getParentFile();
17
18             if (carpeta != null && !carpeta.exists()) {
19                 carpeta.mkdirs();
20             }
21
22             try (PrintWriter writer = new PrintWriter(new
23                 FileWriter(file, true))) {
24                 if (archivoNuevo) {
25                     writer.println("timestamp,nodo_id,nodo,tipo,
26                         detalle");
27                 }
28             }
29         } catch (IOException e) {
30             e.printStackTrace();
31         }
32     }
33 }

```

```

24         }
25
26         writer.println(timestamp + ","
27             + nodoId + ","
28             + nodo + ","
29             + tipo + ","
30             + detalle.replace(",", " "));
31     }
32
33     } catch (IOException e) {
34         System.out.println("Error al guardar evento CSV: " + e
35             .getMessage());
36     }
37 }

```

4.5. AuthToken.java

Esta clase contiene un token de seguridad usado para validar operaciones. Permite rechazar mensajes de tipo OPERACION cuando no tienen un token válido.

```

1  package org.utecq.common;
2
3  public class AuthToken {
4
5      public static final String TOKEN_VALIDO = "TOKEN_SEGURO_123";
6
7      public static boolean esValido(String token) {
8          return TOKEN_VALIDO.equals(token);
9      }
10 }

```

5. Creación de la comunicación TCP

5.1. ServidorTCP.java

Esta clase inicia el servidor TCP de cada nodo. Escucha conexiones, recibe mensajes, actualiza el reloj Lamport, responde heartbeats, detecta fallos e inicia el algoritmo Bully si cae el coordinador.

```

1  package org.utecq.tcp;
2
3  import com.google.gson.Gson;
4  import org.utecq.common.AuthToken;

```



```

5 import org.uteq.common.CsvLogger;
6 import org.uteq.common.Mensaje;
7 import org.uteq.common.Nodo;
8 import org.uteq.common.RelojLamport;
9
10 import java.io.BufferedReader;
11 import java.io.InputStreamReader;
12 import java.io.PrintWriter;
13 import java.net.ServerSocket;
14 import java.net.Socket;
15 import java.util.HashSet;
16 import java.util.Set;
17
18 public class ServidorTCP {
19
20     private static final Gson gson = new Gson();
21
22     private final Nodo nodo;
23     private final RelojLamport reloj;
24
25     private final Set<Integer> nodosCaidos = new HashSet<>();
26     private int coordinadorActual = 3;
27
28     public ServidorTCP(Nodo nodo) {
29         this.nodo = nodo;
30         this.reloj = new RelojLamport();
31     }
32
33     public void iniciar() {
34         iniciarHeartbeat();
35
36         try (ServerSocket servidor = new ServerSocket(nodo.
37             getPuertoTcp())) {
38             System.out.println(nodo.getNombre() + " escuchando por
39                 TCP en puerto " + nodo.getPuertoTcp());
40             System.out.println("Coordinador inicial esperado:
41                 Nodo3");
42
43             while (true) {
44                 Socket socket = servidor.accept();
45                 new Thread(() -> atenderCliente(socket)).start();
46             }
47
48         } catch (Exception e) {
49             System.out.println("Error en servidor TCP " + nodo.
50                 getNombre() + ": " + e.getMessage());
51         }
52     }
53 }

```

```

48     }
49
50     private void atenderCliente(Socket socket) {
51         try (
52             BufferedReader entrada = new BufferedReader(new
                    InputStreamReader(socket.getInputStream()));
53             PrintWriter salida = new PrintWriter(socket.
                    getOutputStream(), true)
54         ) {
55             String jsonRecibido = entrada.readLine();
56
57             if (jsonRecibido == null || jsonRecibido.trim().
                    isEmpty()) {
58                 return;
59             }
60
61             Mensaje mensaje = gson.fromJson(jsonRecibido, Mensaje.
                    class);
62
63             if ("OPERACION".equals(mensaje.getTipo()) && !
                    AuthToken.esValido(mensaje.getToken())) {
64                 Mensaje error = new Mensaje(
65                     "ERROR",
66                     nodo.getNombre(),
67                     nodo.getId(),
68                     reloj.incrementar(),
69                     "Operacion rechazada: token invalido",
70                     null
71                 );
72
73                 salida.println(gson.toJson(error));
74                 return;
75             }
76
77             int relojActualizado = reloj.actualizar(mensaje.
                    getTimestamp());
78
79             if ("HEARTBEAT".equals(mensaje.getTipo())) {
80                 responderHeartbeat(salida);
81                 return;
82             }
83
84             if ("ELECCION".equals(mensaje.getTipo())) {
85                 responderEleccion(salida, mensaje);
86                 return;
87             }
88

```

```

89         if ("COORDINADOR".equals(mensaje.getTipo())) {
90             coordinadorActual = mensaje.getNodeId();
91
92             System.out.println "[" + nodo.getNombre() + "]
                Nuevo coordinador recibido: Nodo" +
                coordinadorActual);
93
94             Mensaje respuesta = new Mensaje(
95                 "ACK",
96                 nodo.getNombre(),
97                 nodo.getId(),
98                 reloj.incrementar(),
99                 "Coordinador actualizado en " + nodo.
                getNombre(),
100                 null
101             );
102
103             salida.println(gson.toJson(respuesta));
104             return;
105         }
106
107         System.out.println "[" + nodo.getNombre() + "] Mensaje
                recibido");
108         System.out.println "Tipo: " + mensaje.getTipo();
109         System.out.println "Remitente: " + mensaje.getSender()
                );
110         System.out.println "Timestamp recibido: " + mensaje.
                getTimestamp());
111         System.out.println "Reloj local actualizado: " +
                relojActualizado);
112         System.out.println "Mensaje: " + mensaje.getMessage()
                ;
113         System.out.println "Coordinador actual: Nodo" +
                coordinadorActual);
114         System.out.println (
                "-----");
115
116         CsvLogger.guardarEvento(
117             "src/main/resources/resultados/eventos_tcp.csv
                ",
118             relojActualizado,
119             nodo.getId(),
120             nodo.getNombre(),
121             mensaje.getTipo(),
122             mensaje.getMessage()
123         );
124

```

```

125         Mensaje respuesta = new Mensaje(
126             "ACK",
127             nodo.getNombre(),
128             nodo.getId(),
129             reloj.incrementar(),
130             "ACK recibido por " + nodo.getNombre(),
131             null
132         );
133
134         salida.println(gson.toJson(respuesta));
135
136     } catch (Exception e) {
137         System.out.println("Error atendiendo cliente en " +
138             nodo.getNombre() + ": " + e.getMessage());
139     }
140
141     private void responderHeartbeat(PrintWriter salida) {
142         Mensaje respuesta = new Mensaje(
143             "ACK",
144             nodo.getNombre(),
145             nodo.getId(),
146             reloj.incrementar(),
147             "Heartbeat OK desde " + nodo.getNombre(),
148             null
149         );
150
151         salida.println(gson.toJson(respuesta));
152     }
153
154     private void responderEleccion(PrintWriter salida, Mensaje
155         mensaje) {
156         Mensaje respuesta = new Mensaje(
157             "ACK",
158             nodo.getNombre(),
159             nodo.getId(),
160             reloj.incrementar(),
161             "Nodo " + nodo.getId() + " responde a eleccion de
162                 Nodo " + mensaje.getNodoId(),
163             null
164         );
165
166         salida.println(gson.toJson(respuesta));
167
168         if (nodo.getId() > mensaje.getNodoId()) {
169             iniciarEleccionBully();
170         }

```

```

169     }
170
171     private void iniciarHeartbeat() {
172         new Thread(() -> {
173             while (true) {
174                 try {
175                     Thread.sleep(3000);
176
177                     for (Nodo otroNodo : Nodo.TODOS) {
178                         if (otroNodo.getId() == nodo.getId()) {
179                             continue;
180                         }
181
182                         Mensaje heartbeat = new Mensaje(
183                             "HEARTBEAT",
184                             nodo.getNombre(),
185                             nodo.getId(),
186                             reloj.incrementar(),
187                             "Latido desde " + nodo.getNombre()
188                             ,
189                             null
190                         );
191
192                         Mensaje respuesta = ClienteTCP.
193                             enviarMensaje(otroNodo, heartbeat);
194
195                         if (respuesta == null) {
196                             if (!nodosCaidos.contains(otroNodo.
197                                 getId())) {
198                                 nodosCaidos.add(otroNodo.getId());
199
200                                 System.out.println "[" + nodo.
201                                     getNombre() + "]" Detecto caida
202                                     de " + otroNodo.getNombre());
203
204                                 CsvLogger.guardarEvento(
205                                     "src/main/resources/
206                                         resultados/eventos_tcp.
207                                         csv",
208                                     reloj.incrementar(),
209                                     nodo.getId(),
210                                     nodo.getNombre(),
211                                     "FALLO",
212                                     "Se detecto caida de " +
213                                         otroNodo.getNombre()
214                                 );
215                             }
216                         }
217                     }
218                 } catch (InterruptedException e) {
219                     // Ignorar
220                 }
221             }
222         });
223     }

```

```

208         if (otroNodo.getId() ==
                coordinadorActual) {
209             System.out.println "[" + nodo.
                    getNombre() + "] Cayo el
                    coordinador. Iniciando
                    Bully...");
                iniciarEleccionBully();
210         }
211     }
212 }
213 } else {
214     nodosCaidos.remove(otroNodo.getId());
215 }
216 }
217
218 } catch (Exception e) {
219     System.out.println("Error en heartbeat de " +
        nodo.getNombre() + ": " + e.getMessage());
220 }
221 }
222 }).start();
223 }
224
225 private void iniciarEleccionBully() {
226     boolean existeNodoMayorActivo = false;
227
228     for (Nodo otroNodo : Nodo.TODOS) {
229         if (otroNodo.getId() > nodo.getId() && !nodosCaidos.
            contains(otroNodo.getId())) {
230             Mensaje eleccion = new Mensaje(
231                 "ELECCION",
232                 nodo.getNombre(),
233                 nodo.getId(),
234                 reloj.incrementar(),
235                 "Eleccion iniciada por " + nodo.getNombre()
                    (),
236                 null
237             );
238
239             Mensaje respuesta = ClienteTCP.enviarMensaje(
                otroNodo, eleccion);
240
241             if (respuesta != null) {
242                 existeNodoMayorActivo = true;
243                 System.out.println "[" + nodo.getNombre() + "]
                    Nodo mayor activo respondio: " + otroNodo.
                        getNombre());
244             } else {

```

```

245         nodosCaidos.add(otroNodo.getId());
246     }
247 }
248 }
249
250 if (!existeNodoMayorActivo) {
251     coordinadorActual = nodo.getId();
252
253     System.out.println("[ " + nodo.getNombre() + " ] Se
        declara nuevo coordinador");
254
255     CsvLogger.guardarEvento(
256         "src/main/resources/resultados/eventos_tcp.csv",
257         reloj.incrementar(),
258         nodo.getId(),
259         nodo.getNombre(),
260         "COORDINADOR",
261         nodo.getNombre() + " se declara coordinador"
262     );
263
264     notificarCoordinador();
265 }
266 }
267
268 private void notificarCoordinador() {
269     for (Nodo otroNodo : Nodo.TODOS) {
270         if (otroNodo.getId() == nodo.getId() || nodosCaidos.
            contains(otroNodo.getId())) {
271             continue;
272         }
273
274         Mensaje coordinador = new Mensaje(
275             "COORDINADOR",
276             nodo.getNombre(),
277             nodo.getId(),
278             reloj.incrementar(),
279             "Nuevo coordinador: " + nodo.getNombre(),
280             null
281         );
282
283         ClienteTCP.enviarMensaje(otroNodo, coordinador);
284     }
285 }
286 }

```

5.2. ClienteTCP.java

Esta clase permite enviar mensajes TCP desde un nodo hacia otro. Convierte los mensajes a JSON, los envía por socket y recibe la respuesta del nodo destino.

```
1 package org.uteq.tcp;
2
3 import com.google.gson.Gson;
4 import org.uteq.common.Mensaje;
5 import org.uteq.common.Nodo;
6
7 import java.io.BufferedReader;
8 import java.io.InputStreamReader;
9 import java.io.PrintWriter;
10 import java.net.Socket;
11
12 public class ClienteTCP {
13
14     private static final String HOST = "localhost";
15     private static final Gson gson = new Gson();
16
17     public static Mensaje enviarMensaje(Nodo destino, Mensaje
18         mensaje) {
19         try (
20             Socket socket = new Socket(HOST, destino.
21                 getPuertoTcp());
22             PrintWriter salida = new PrintWriter(socket.
23                 getOutputStream(), true);
24             BufferedReader entrada = new BufferedReader(new
25                 InputStreamReader(socket.getInputStream()))
26         ) {
27             String json = gson.toJson(mensaje);
28
29             // Delimitacion por linea: cada mensaje termina con
30             // println.
31             salida.println(json);
32
33             String respuestaJson = entrada.readLine();
34
35             if (respuestaJson == null) {
36                 return null;
37             }
38
39             return gson.fromJson(respuestaJson, Mensaje.class);
40         } catch (Exception e) {
41             System.out.println("No se pudo enviar mensaje TCP a "
```



```

38         + destino.getNombre()
39         + " puerto "
40         + destino.getPuertoTcp()
41         + ": "
42         + e.getMessage());
43
44     return null;
45 }
46 }
47 }

```

6. Creación de los nodos distribuidos

6.1. *Nodo1TCP.java*

Esta clase inicia el servidor correspondiente a Nodo1, utilizando el puerto 5000.

```

1  package org.uteq.tcp.nodos;
2
3  import org.uteq.common.Nodo;
4  import org.uteq.tcp.ServidorTCP;
5
6  public class Nodo1TCP {
7
8      public static void main(String[] args) {
9          ServidorTCP servidor = new ServidorTCP(Nodo.NOD01);
10         servidor.iniciar();
11     }
12 }

```

6.2. *Nodo2TCP.java*

Esta clase inicia el servidor correspondiente a Nodo2, utilizando el puerto 5001.

```

1  package org.uteq.tcp.nodos;
2
3  import org.uteq.common.Nodo;
4  import org.uteq.tcp.ServidorTCP;
5
6  public class Nodo2TCP {
7
8      public static void main(String[] args) {
9          ServidorTCP servidor = new ServidorTCP(Nodo.NOD02);
10         servidor.iniciar();
11     }

```

```
12 }
```

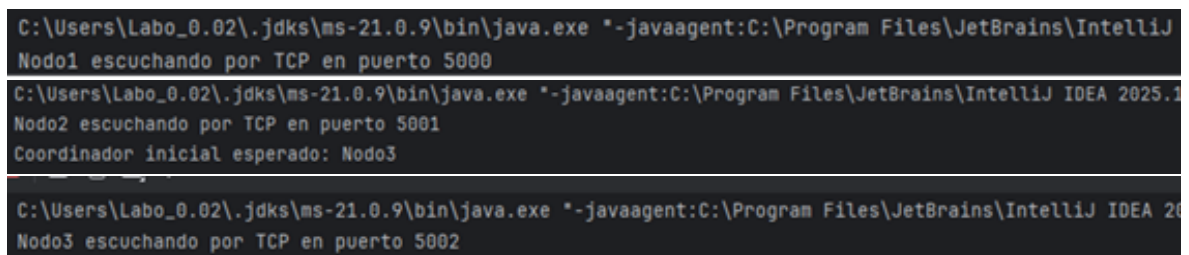
6.3. *Nodo3TCP.java*

Esta clase inicia el servidor correspondiente a Nodo3, utilizando el puerto 5002, en la cual, en el prototipo, Nodo3 se toma como coordinador inicial.

```
1 package org.uteq.tcp.nodos;  
2  
3 import org.uteq.common.Nodo;  
4 import org.uteq.tcp.ServidorTCP;  
5  
6 public class Nodo3TCP {  
7  
8     public static void main(String[] args) {  
9         ServidorTCP servidor = new ServidorTCP(Nodo.NOD03);  
10        servidor.iniciar();  
11    }  
12 }
```

7. Evidencia de ejecución

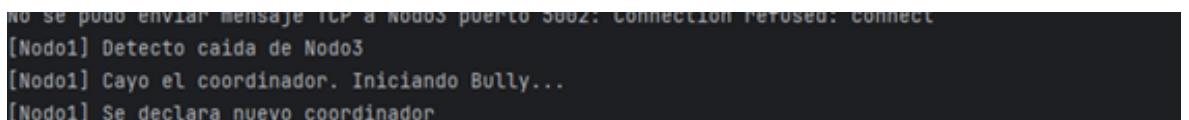
7.1. *Ejecución de los tres nodos*



```
C:\Users\Labo_0.02\.jdk\ms-21.0.9\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ  
Nodo1 escuchando por TCP en puerto 5000  
C:\Users\Labo_0.02\.jdk\ms-21.0.9\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.1  
Nodo2 escuchando por TCP en puerto 5001  
Coordinador inicial esperado: Nodo3  
C:\Users\Labo_0.02\.jdk\ms-21.0.9\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 20  
Nodo3 escuchando por TCP en puerto 5002
```

Figura 3: Ejecución de los tres nodos

7.2. *Prueba de caída del coordinador*



```
No se pudo enviar mensaje TCP a Nodos puerto 5002: Connection refused: connect  
[Nodo1] Detecto caída de Nodo3  
[Nodo1] Cayo el coordinador. Iniciando Bully...  
[Nodo1] Se declara nuevo coordinador
```

Figura 4: Prueba de caída del coordinador

7.3. Elección del nuevo coordinador N2

```
[Nodo2] Detecto caída de Nodo3  
[Nodo2] Cayo el coordinador. Iniciando Bully...  
[Nodo2] Se declara nuevo coordinador  
No se pudo enviar mensaje TCP a Nodo3 puerto 5002: Connection refused: connect
```

Figura 5: Elección del nuevo coordinador N2

7.4. Evidencia del archivo ANALISIS.md

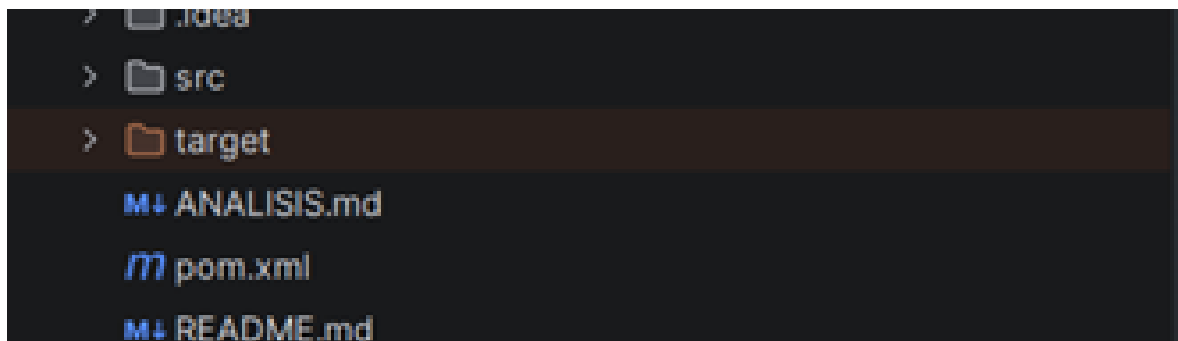


Figura 6: Evidencia del archivo ANALISIS.md

CONCLUSIÓN

Se implementó un prototipo distribuido en Java con tres nodos TCP, aplicando comunicación por sockets, reloj Lamport, heartbeats y algoritmo Bully, en donde las pruebas permitieron evidenciar que los nodos pueden comunicarse, detectar la caída del coordinador y elegir un nuevo líder para continuar funcionando. Además, se registraron los eventos en CSV como evidencia de la ejecución.